HotelPro – Functional Specification Document

1. Introduction

- Project title: HotelPro – Hotel Management System (MERN stack)
- Team members: SWUID2026403463

2. Project Overview

Purpose. HotelPro lets guests find a room, choose their dates and pay online, while front desk staff, managers and admins run rooms, bookings, guest records and reports from a separate panel.

Guest features

- Register, verify by OTP, log in, reset password and edit profile
- Browse, search and filter rooms by dates, guests, room type, price and amenities, with sorting
- Room page with photo gallery, amenities, rates, reviews and availability by date
- Room hold with a 10-minute countdown, no double booking, and guests limited to the room capacity
- Add services (breakfast, airport pickup) and a coupon, with taxes and total shown
- Simulated payment by card, UPI, net banking or wallet
- Booking confirmation with booking ID and invoice, printable; booking history; cancellation up to 24 hours before check-in with simulated refund
- Star ratings and reviews after a completed stay; notifications by email and in-app

Staff features (at /staff/login)

- Front desk staff: check guests in and out, keep guest records, view bookings
- Hotel manager: dashboard (revenue, bookings, occupancy, last 7 days), manage rooms with photos, rates and services, occupancy reports
- Admin: activate, deactivate or delete users and staff accounts, manage coupons, view and cancel all bookings, process refunds, view the audit log

3. Architecture

Frontend (React + Vite). A single-page app in plain JavaScript/JSX using React Router v6, styled with Bootstrap and custom CSS. App.jsx defines the routes; RequireAuth guards guest pages (role="guest") and the staff area (role="staff", with manager and admin checks per page). Login state lives in AuthContext, API calls go through api.js, and shared UI sits in components/ (Navbar, Footer, RoomCard, PhotoGallery, ui.jsx). The Vite dev server proxies /api to port 5000, so the browser sees one origin and the login cookie works without CORS setup.

Backend (Node.js + Express). server.js configures the app (helmet, CORS, JSON parser, cookie-parser, morgan, rate limiting on login/register, multer for room photo uploads) and mounts eight route groups under /api: auth, staff, rooms, bookings, guests, reviews, services and coupons. Controllers hold the logic; middleware handles authentication, role checks, ObjectId validation and centralised errors. utils/ holds the availability logic, token generation, the simulated payment gateway, email sending and the seed script.

Database (MongoDB with Mongoose). Nine collections:

Collection	Key fields	Relations
User	name, email (unique), hashed password, phone, isVerified, isActive	–
Staff	name, email (unique), hashed password, role (staff / manager / admin), isActive	–
Room	roomNumber (unique), type, description, pricePerNight, capacity, amenities[], photos[] (binary data + contentType, stored in the database), isActive	–
Service	name, price, isActive	–
Coupon	code (unique), discountPercent, maxDiscount, expiresAt, isActive	–
Booking	bookingCode, checkIn, checkOut, guests, services[] (name, price), couponCode, totalAmount, status (pending / confirmed / checked-in / completed / cancelled / expired), expiresAt, payment (method, status, transactionId, paidAt)	user, room

Collection	Key fields	Relations
GuestRecord	idType, idNumber, checkedInAt, checkedOutAt, notes	booking, user
Review	rating 1–5, comment (max 1000 chars); one per completed booking	user, room, booking
AuditLog	action, target, createdAt	staff (actor)

4. Setup Instructions

Prerequisites

- Node.js 18 or newer (check with node -v)
- MongoDB: either MongoDB Community Server installed locally, or a free MongoDB Atlas cluster whose connection string goes in MONGO_URI

Installation

1. Extract the project and open two terminals, one in server/ and one in client/.
2. In server/, run npm install.
3. Open server/.env and set the variables below (change JWT_SECRET to a long random string).
4. Run npm run seed to create the admin account, sample rooms, services and a coupon (it only seeds an empty database).
5. In client/, run npm install.

Variable (server/.env)	Purpose
PORT	API port (default 5000)
NODE_ENV	development or production (production makes cookies secure; sameSite=none)
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret used to sign tokens
JWT_EXPIRES_IN	Token lifetime (default 7d)

Variable (server/.env)	Purpose
CLIENT_URL	Allowed CORS origin (default http://localhost:5173)
ROOM_HOLD_MINUTES	Room hold while payment is pending (default 10)
ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD	Default admin created by the seed script
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, EMAIL_FROM	Optional email settings; if SMTP_HOST is empty, emails print in the server console

Optional in client/: VITE_API_URL is needed only when the API is hosted on a different origin.

5. Folder Structure

Client (client/)

```
client/
  index.html, vite.config.js, package.json, .env.example
  public/favicon.svg
  src/
    main.jsx, App.jsx            entry point and routes
    api.js, hooks.js, utils.js
    index.css
    context/AuthContext.jsx      login state
    components/                  Navbar, Footer, RoomCard, PhotoGallery,
ui.jsx
    pages/                       Home, Rooms, RoomDetail, Booking, Checkout,
                                 BookingDetail, MyBookings, Profile, Login,
Register
    pages/staff/                 StaffLogin, StaffLayout, Dashboard,
ManageRooms,
                                 ManageBookings, CheckInOut, GuestRecords,
                                 Reports, ManageUsers
```

Server (server/)

```
server/
  index.js          entry point: loads .env, connects DB, starts server
  server.js         Express app, middleware and route mounting
  config/db.js      MongoDB connection
  models/           user, staff, room, service, coupon, booking, guestRecord,
review, auditLog
  controllers/      auth, staff, room, booking, guest, review, service,
coupon
  routes/           one router per controller
  middleware/       authMiddleware, upload, validateObjectId, errorMiddleware
  utils/            availabilityUtils, generateToken, paymentGateway,
sendEmail, seed, AppError, asyncHandler
```

6. Running the Application

Start the backend first, then the frontend, each in its own terminal.

Part	Directory	Command	Address
Backend (development)	server/	npm run dev (nodemon)	http://localhost:5000
Backend (plain)	server/	npm start	http://localhost:5000
Frontend (development)	client/	npm run dev	http://localhost:5173

The project uses Vite, so the frontend command is npm run dev rather than npm start. Check the API at /api/health. For production, run npm run build in client/ and serve client/dist from any static host.

Role	Login URL	Credentials
Guest	/register, /login	any email and password you create
Staff / Manager / Admin	/staff/login	admin@hotelpro.com / Admin@123 (seeded demo account)

7. API Documentation

Base path /api. Every response is JSON with a success flag; errors return { "success": false, "message": "..." }. Access: Public = no login, Guest = guest login, Staff = any staff-side login (front desk, manager or admin), Manager = manager or admin, Admin = admin only, Any = guest or staff.

Method	Endpoint	Access	Purpose / parameters
GET	/health	Public	API status check
POST	/auth/register	Public	Create account: name, email, password, phone; sends OTP
POST	/auth/verify-otp	Public	Verify account: email, otp
POST	/auth/login	Public	Guest login: email, password
POST	/auth/forgot-password, /auth/reset-password	Public	Request a reset OTP / set a new password
POST	/auth/logout	Public	Clear the login cookie
GET	/auth/me	Any	Current account
PUT	/auth/me	Any	Update profile / change password
POST	/staff/login	Public	Staff, manager and admin login: email, password
GET	/staff/dashboard	Manager	Revenue, bookings, occupancy, last 7 days
GET	/staff/reports/occupancy	Manager	Occupancy and revenue report. Query: from, to
GET	/staff/users, /staff/users/:id	Admin	List / view guests and staff
POST, PUT, DELETE	/staff/users, /staff/users/:id	Admin	Create a staff account / activate or deactivate / delete a user
GET	/staff/bookings	Staff	All bookings

Method	Endpoint	Access	Purpose / parameters
GET	/staff/audit-log	Admin	Audit log of staff actions
GET	/rooms	Public	Query: search, type, minPrice, maxPrice, amenities, checkIn, checkOut, guests, sort, page, limit
GET	/rooms/:id	Public	Room details with availability. Query: checkIn, checkOut
GET	/rooms/:id/photos/:photoId	Public	Room photo served from the database
POST, PUT, DELETE	/rooms, /rooms/:id	Manager	Create / update / delete a room (photos sent as multipart/form-data)
GET	/services	Public	List services (breakfast, airport pickup and others)
POST, PUT, DELETE	/services, /services/:id	Manager	Manage services
POST	/coupons/validate	Guest	Check a coupon code: code, bookingTotal
POST, PUT, DELETE	/coupons, /coupons/:id	Admin	Manage coupons
POST	/bookings	Guest	Hold a room (status pending): room, checkIn, checkOut, guests, services, coupon
POST	/bookings/:id/pay	Guest	Pay: method (card, upi, netbanking, wallet) and details
GET	/bookings/my	Guest	Own bookings
GET	/bookings/:id	Any	One booking (owner or staff)

Method	Endpoint	Access	Purpose / parameters
PUT	/bookings/:id/cancel	Any	Cancel up to 24 hours before check-in
PUT	/bookings/:id/check-in	Staff	Check a guest in
PUT	/bookings/:id/check-out	Staff	Check a guest out and close the booking
GET, POST	/guests, /guests/:id	Staff	Search / add / view guest records (ID details)
GET	/reviews/room/:roomId	Public	Reviews for a room
POST	/reviews	Guest	Add review after a completed stay: booking, rating (1–5), comment
PUT	/reviews/:id	Guest	Edit own review
DELETE	/reviews/:id	Any	Delete (owner or admin)

Example: login

```
POST /api/auth/login
{ "email": "guest@example.com", "password": "secret123" }

200 OK
{
  "success": true,
  "token": "<jwt>",
  "user": { "_id": "...", "name": "Asha", "email": "guest@example.com",
"role": "guest", "phone": "..." }
}
```

Example: cancel a booking

```
PUT /api/bookings/:id/cancel

200 OK
{ "success": true, "message": "Booking cancelled. Refund initiated.",
"booking": { ... } }

400 Bad Request (less than 24 hours before check-in)
{ "success": false, "message": "Bookings can only be cancelled up to 24 hours
before check-in" }
```

Booking flow. POST /bookings holds the room for the chosen dates for 10 minutes; POST /bookings/:id/pay confirms the booking. If payment is not made in time the booking becomes expired and the room is released. The mock gateway rejects a card number ending in 1111, requires a 16-digit card number and a name@bank UPI id, and always accepts net banking and wallet.

8. Authentication

- Passwords are hashed with bcrypt (10 rounds) before saving and are excluded from queries by default.
- **Tokens.** On login or registration the server signs a JWT containing the account id and role (guest, staff, manager or admin), valid for 7 days by default (JWT_EXPIRES_IN).
- **Cookie.** The token is sent in an httpOnly cookie named token (secure and sameSite=none in production, sameSite=lax in development), so scripts in the page cannot read it. An Authorization: Bearer <token> header is also accepted.
- **Authorization.** protect verifies the token and loads the account (rejecting deleted or deactivated users); guestOnly, staffOnly, managerOnly and adminOnly enforce roles. Staff have their own collection and a separate login endpoint and page.
- **Frontend.** AuthContext loads the account from /auth/me, and RequireAuth redirects visitors away from pages that need a login or a specific role.
- **Hardening.** helmet headers, CORS limited to CLIENT_URL, and a limit of 100 login/register attempts per IP every 15 minutes.

9. User Interface

Screens are listed with their routes. Screenshots or GIFs can be dropped in under each row once the app is running.

Screen	Route	Access	What it shows
Home	/	Public	Featured rooms and search by dates and guests
Rooms	/rooms	Public	Search, filters and sorting with room cards
Room detail	/rooms/:id	Public	Photo gallery, amenities, rates, reviews, availability by date
Booking	/rooms/:id/book	Guest	Dates, guests, services, coupon and booking summary
Checkout	/checkout/:id	Guest	Hold countdown and payment method form
Booking detail	/bookings/:id	Guest	Confirmation with booking ID and invoice, printable
My bookings	/my-bookings	Guest	History and cancellation
Profile	/profile	Guest	Edit details, change password
Login / Register	/login, /register	Public	Guest authentication
Staff login	/staff/login	Public	Staff, manager and admin authentication
Staff panel	/staff	Staff	Dashboard, rooms and photos, bookings, check-in/out, guest records, reports, users (by role)

10. Testing

Automated tests are not part of v1.0.0; the main flows are checked by hand using the UAT template (TC-001 to TC-040). Tools and checks available:

- **Manual end-to-end testing** of register, login, room search, room hold, payment, invoice, cancel, check-in/out and review, plus every staff screen.
- **Seeded demo data** (npm run seed) gives a repeatable starting state.
- **Payment test cases:** a card ending 1111 is declined; invalid card numbers or UPI ids are rejected; net banking and wallet succeed.
- **API checks** with Postman or curl against /api/health and the endpoints in section 7.

- **ESLint** (npm run lint in client/) for static checks.

11. Screenshots or Demo

Screenshots are added here once the app is running locally.

- Home Page: [add screenshot]
- Login Page: [add screenshot]
- Room Detail Page: [add screenshot]
- Booking Page: [add screenshot]
- Checkout Page: [add screenshot]
- Confirmation Page: [add screenshot]
- Staff Dashboard: [add screenshot]

12. Known Issues

- Payments are simulated: no real gateway is connected and refunds are only marked as initiated.
- Emails are printed to the server console unless the SMTP_* variables are set.
- Room photos are stored in MongoDB, so large images increase the database size; keep photos small.
- Port 5000 may already be in use (for example macOS AirPlay); change PORT in server/.env and the proxy target in client/vite.config.js.
- The seeded admin password is public in the README and must be changed before any real deployment.
- No automated tests exist.

13. Future Enhancements

- Real payment gateway (for example Razorpay or Stripe) with actual refunds
- Automated unit and API tests, and CI
- SMS and email booking confirmation with a QR code
- Cloud image storage instead of storing photos in the database
- Housekeeping and maintenance management
- Seasonal pricing, offers and room recommendations based on past bookings

- Multi-language interface and a mobile app